FACULTY OF ENGINEERING AT SHOUBRA

Benha University

Course: CCE414 & ELE355 - Artificial Intelligence

EXPERT SYSTEM PROJECT (Project 2)

Project Title: Car Diagnosis Expert System

Team Members:

1. Ramy Said Eliwa Elsayed Mohammed (ID:231903795)
2. Ahmed Mohammed Abdul Razik (ID:232903959)
3. David George Helmy Ekladious Shenoda (ID:231903654)

Task Distribution Table:

Team Member	Assigned Tasks
Ramy Said Eliwa Elsayed Mohammed	Knowledge Engineering: Knowledge Base Formulation (Rules & Facts), System Testing, and Debugging.
Ahmed Mohammed Abdul Razik	Documentation & Design: Algorithm Design, System Flowchart Creation, and Final Report Formatting/Compilation.
David George Helmy Ekladious Shenoda	Core Programming: Prolog System Design, Logic Implementation (Backtracking & Cut), and User Interface Setup.

1. Introduction

Problem Statement:

Car owners often experience sudden vehicle malfunctions but lack the mechanical expertise to identify the root cause. This leads to unnecessary stress, potential safety hazards, and reliance on costly diagnostic services for simple issues.

Purpose of the Expert System:

The purpose of this project is to design and develop an expert system that diagnoses common vehicle problems based on symptoms described by the user. By asking simple yes/no questions, the system identifies the fault and suggests possible repairs or maintenance actions.

Users (Clients) of the System:

The primary users are everyday car owners, drivers, and novice mechanics who need immediate troubleshooting assistance.

The Expert(s):

The system emulates the diagnostic reasoning of an experienced auto-mechanic.

Resources Used:

- **Software:** SWI-Prolog for programming the knowledge base and inference engine.
- **Domain Knowledge:** General automotive maintenance guides, vehicle troubleshooting manuals, and domain expertise modeled after mechanic diagnostic trees.

2. Body of the Report

A. Technique Used to Acquire Knowledge

The knowledge acquisition process involved manual extraction from automotive troubleshooting charts and manuals. We mapped observable symptoms (e.g., "dim lights", "engine misfires") to specific mechanical failures (e.g., "battery", "spark plugs"). This knowledge was then translated into a rule-based format using Prolog, establishing a Knowledge Base expert system rather than a standard algorithmic flow.

B. List of Rules

The system successfully implements 10 distinct rules for diagnosis:

1. **Battery:** IF the engine won't start AND the lights are dim, THEN the problem is the battery.
2. **Spark Plugs:** IF the engine misfires AND there is high fuel consumption, THEN the problem is the spark plugs.
3. **Radiator:** IF the car is overheating AND leaking coolant, THEN the problem is the radiator.
4. **Brake Pads:** IF there are squealing brakes AND poor braking performance, THEN the problem is the brake pads.
5. **Flat Tire:** IF there is a flapping noise AND the car pulls to the side, THEN the problem is a flat tire.
6. **Fuel Pump:** IF the engine stalls AND there is a whining noise, THEN the problem is the fuel pump.
7. **Oxygen Sensor:** IF the check engine light is on AND there is black smoke, THEN the problem is the oxygen sensor.
8. **Alternator:** IF the battery light is on AND there are electrical issues, THEN the problem is the alternator.
9. **Transmission:** IF there is hard shifting AND leaking red fluid, THEN the problem is the transmission.
10. **Worn Wiper:** IF there is poor visibility AND streaking on the glass, THEN the problem is worn wiper blades.

C. Detailed Code Explanation and Architecture

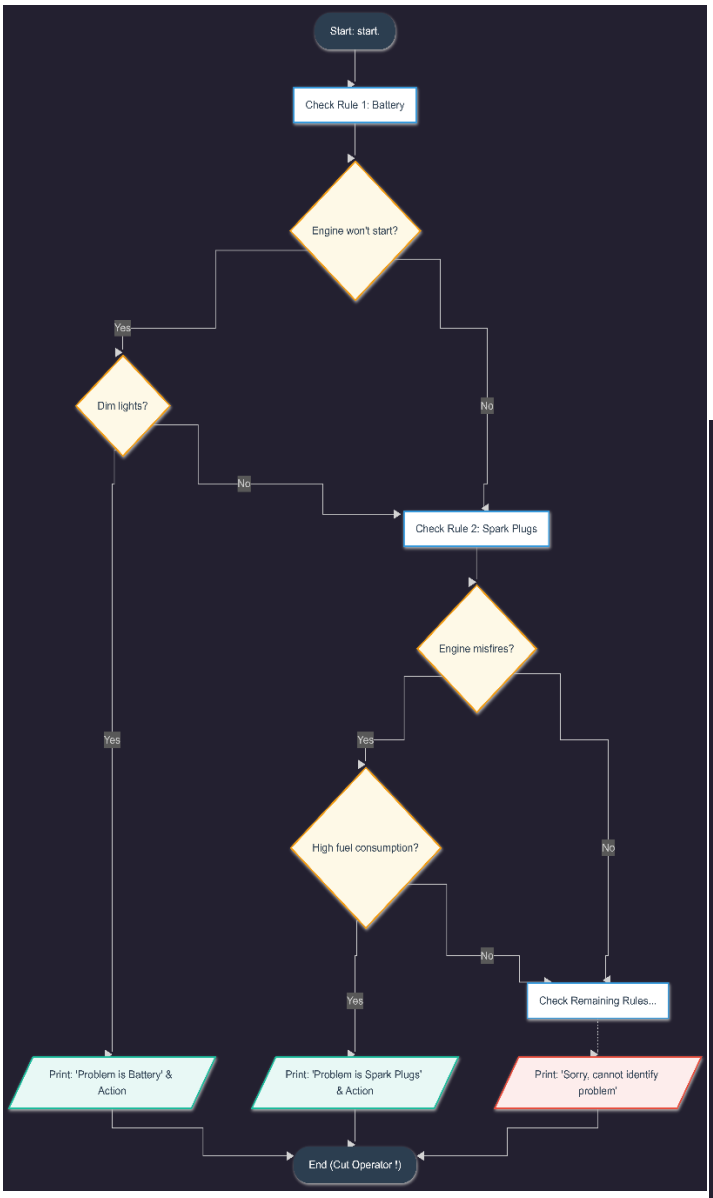
The system is built entirely in Prolog and utilizes backward chaining, recursion, and backtracking to traverse the knowledge base. Below is a detailed breakdown of the system's core components:

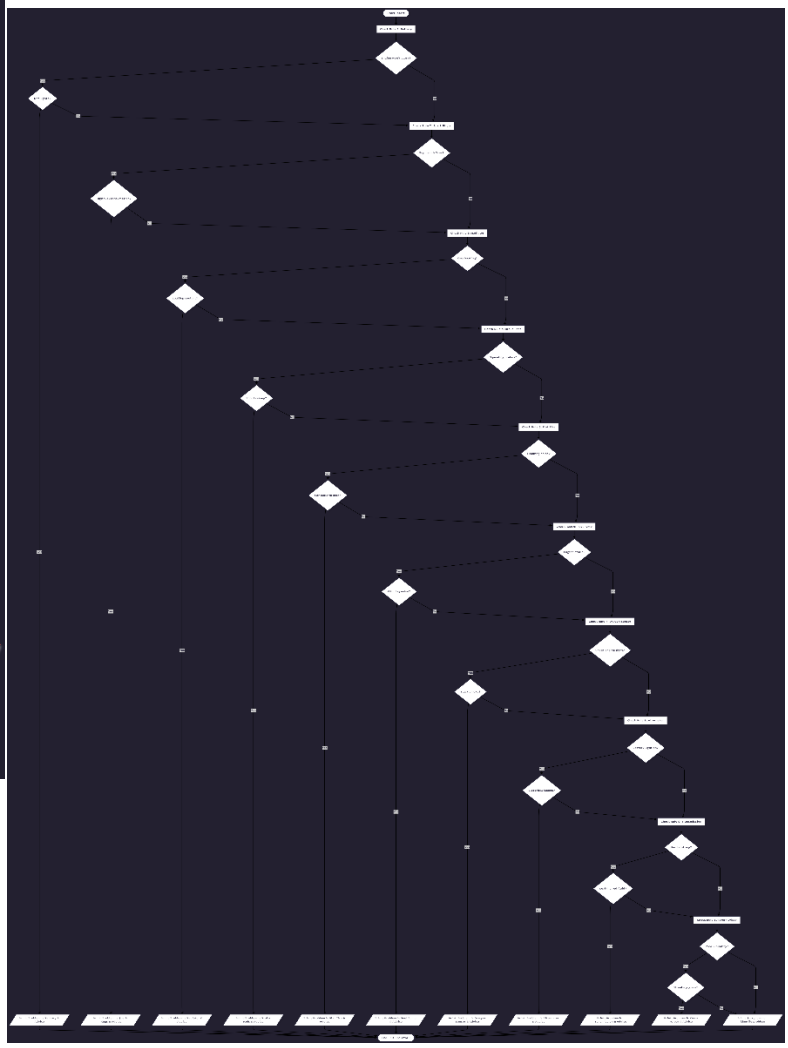
- **1. The Main Control Block (start Predicate):** This is the entry point of the application. It handles the user interface by printing a welcoming message and instructions. After calling diagnose (Problem), the system uses the **Cut operator (!)**. This is a crucial efficiency mechanism. Once the inference engine successfully matches a problem, the Cut prevents Prolog from backtracking and searching for more solutions. A fallback rule is also provided if the system cannot match any rules.
- **2. The Diagnostic Rules (diagnose Predicates):** This section contains the core logic (the IF-THEN rules). Each diagnose rule represents a specific car part failure. Both conditions must return true for the rule to succeed.
- **3. The Solution Base (solution Facts):** Acts as a database mapping known problems to actionable advice.
- **4. The User Input Handler (ask Predicate):** This dynamic function handles all user interaction, waiting for a y. or n. input to confirm symptoms.

D. System Algorithm

1. Initialize the program by calling start.
2. Print the user interface header.
3. Evaluate the diagnose (Problem) rules sequentially.
4. For the current rule, prompt the user with ask (Symptom).
5. Read user input (y. or n.). If y., evaluate the next symptom. If n., Backtrack to the next diagnose rule.
6. Upon a successful rule match, execute Cut (!) to halt backtracking.
7. Output the identified Problem and query/output the solution.
8. If all rules fail, execute the fallback start predicate to inform the user.

E. System Flow Chart





3. Appendices

Appendix A: Complete Source Code

PROJECT.pl

File Edit Browse Compile Prolog Help

PROJECT.pl

```
start :-
    write('*** Car Diagnosis System ***'), nl,
    write('Answer the questions with y. or n.'), nl,
    write('---------------------------------'), nl,
    diagnose(Problem), !,
    write('>> Diagnosis: The problem is '), write(Problem), nl,
    write('>> Advice: '), solution(Problem), nl.

start :-
    write('Sorry, the system cannot identify the problem.'), nl.

diagnose(battery) :- ask(engine_wont_start), ask(dim_lights).
diagnose(spark_plugs) :- ask(engine_misfires), ask(high_fuel_consumption).
diagnose(radiator) :- ask(overheating), ask(leaking_coolant).
diagnose(brake_pads) :- ask(squealing_brakes), ask(poor_braking).
diagnose(flat_tire) :- ask(flapping_noise), ask(car_pulls_to_side).
diagnose(fuel_pump) :- ask(engine_stalls), ask(whining_noise).
diagnose(oxygen_sensor) :- ask(check_engine_light), ask(black_smoke).
diagnose(alternator) :- ask(battery_light_on), ask(electrical_issues).
diagnose(transmission) :- ask(hard_shifting), ask(leaking_red_fluid).
diagnose(worn_wiper) :- ask(poor_visibility), ask(streaking_glass).

solution(battery) :- write('Replace the battery.').
solution(spark_plugs) :- write('Clean or replace spark plugs.').
solution(radiator) :- write('Check hoses and refill coolant.').
```

user:solution/1: (loaded) 10 clauses, 2.2 Kb, primary_index(1) Line: 27

PROJECT.pl

File Edit Browse Compile Prolog Help

PROJECT.pl

```
solution(battery) :- write('Replace the battery.').
solution(spark_plugs) :- write('Clean or replace spark plugs.').
solution(radiator) :- write('Check hoses and refill coolant.').
solution(brake_pads) :- write('Replace brake pads immediately.').
solution(flat_tire) :- write('Change the tire with the spare.').
solution(fuel_pump) :- write('Replace the fuel pump.').
solution(oxygen_sensor) :- write('Replace oxygen sensor.').
solution(alternator) :- write('Check the alternator belt.').
solution(transmission) :- write('Check transmission fluid.').
solution(worn_wiper) :- write('Buy new wiper blades.').

ask(Question) :-
    write('Does the car have ('), write(Question), write(')? (y/n): '),
    read(Reply),
    Reply == y.
```

Appendix B: Output Screenshots

Run 1: Successful Diagnosis (e.g., Battery Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): y.
Does the car have (dim_lights)? (y/n): |: y.
>> Diagnosis: The problem is battery
>> Advice: Replace the battery.
true.
```

Run 2: Successful Diagnosis (e.g., Spark Plugs Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: y.
Does the car have (high_fuel_consumption)? (y/n): |: y.
>> Diagnosis: The problem is spark_plugs
>> Advice: Clean or replace spark plugs.
true.
```

Run 3: Successful Diagnosis (e.g., Radiator Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: y.
Does the car have (leaking_coolant)? (y/n): |: y.
>> Diagnosis: The problem is radiator
>> Advice: Check hoses and refill coolant.
true.
```

Run 4: Successful Diagnosis (e.g., Brake Pads Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: n.
Does the car have (squealing_brakes)? (y/n): |: y.
Does the car have (poor_braking)? (y/n): |: y.
>> Diagnosis: The problem is brake_pads
>> Advice: Replace brake pads immediately.
true.
```

Run 5: Successful Diagnosis (e.g., Flat Tire Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: n.
Does the car have (squealing_brakes)? (y/n): |: n.
Does the car have (flapping_noise)? (y/n): |: y.
Does the car have (car_pulls_to_side)? (y/n): |: y.
>> Diagnosis: The problem is flat_tire
>> Advice: Change the tire with the spare.
true.
```

Run 6: Successful Diagnosis (e.g., Fuel Pump Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: n.
Does the car have (squealing_brakes)? (y/n): |: n.
Does the car have (flapping_noise)? (y/n): |: n.
Does the car have (engine_stalls)? (y/n): |: y.
Does the car have (whining_noise)? (y/n): |: y.
>> Diagnosis: The problem is fuel_pump
>> Advice: Replace the fuel pump.
true.
```

Run 7: Successful Diagnosis (e.g., Oxygen Sensor Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: n.
Does the car have (squealing_brakes)? (y/n): |: n.
Does the car have (flapping_noise)? (y/n): |: n.
Does the car have (engine_stalls)? (y/n): |: n.
Does the car have (check_engine_light)? (y/n): |: y.
Does the car have (black_smoke)? (y/n): |: y.
>> Diagnosis: The problem is oxygen_sensor
>> Advice: Replace oxygen sensor.
true.
```

Run 8: Successful Diagnosis (e.g., Alternator Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: n.
Does the car have (squealing_brakes)? (y/n): |: n.
Does the car have (flapping_noise)? (y/n): |: n.
Does the car have (engine_stalls)? (y/n): |: n.
Does the car have (check_engine_light)? (y/n): |: n.
Does the car have (battery_light_on)? (y/n): |: y.
Does the car have (electrical_issues)? (y/n): |: y.
>> Diagnosis: The problem is alternator
>> Advice: Check the alternator belt.
true.
```

Run 9: Successful Diagnosis (e.g., Transmission Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: n.
Does the car have (squealing_brakes)? (y/n): |: n.
Does the car have (flapping_noise)? (y/n): |: n.
Does the car have (engine_stalls)? (y/n): |: n.
Does the car have (check_engine_light)? (y/n): |: n.
Does the car have (battery_light_on)? (y/n): |: n.
Does the car have (hard_shifting)? (y/n): |: y.
Does the car have (leaking_red_fluid)? (y/n): |: y.
>> Diagnosis: The problem is transmission
>> Advice: Check transmission fluid.
true.
```

Run 10: Successful Diagnosis (e.g., Worn Wiper Issue)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: n.
Does the car have (squealing_brakes)? (y/n): |: n.
Does the car have (flapping_noise)? (y/n): |: n.
Does the car have (engine_stalls)? (y/n): |: n.
Does the car have (check_engine_light)? (y/n): |: n.
Does the car have (battery_light_on)? (y/n): |: n.
Does the car have (hard_shifting)? (y/n): |: n.
Does the car have (poor_visibility)? (y/n): |: y.
Does the car have (streaking_glass)? (y/n): |: y.
>> Diagnosis: The problem is worn_wiper
>> Advice: Buy new wiper blades.
true.
```

Run 11: Unsuccessful Diagnosis (e.g., Fallback / No matching symptoms)

```
6 ?- start.
*** Car Diagnosis System ***
Answer the questions with y. or n.
---------------------------------
Does the car have (engine_wont_start)? (y/n): n.
Does the car have (engine_misfires)? (y/n): |: n.
Does the car have (overheating)? (y/n): |: n.
Does the car have (squealing_brakes)? (y/n): |: n.
Does the car have (flapping_noise)? (y/n): |: n.
Does the car have (engine_stalls)? (y/n): |: n.
Does the car have (check_engine_light)? (y/n): |: n.
Does the car have (battery_light_on)? (y/n): |: n.
Does the car have (hard_shifting)? (y/n): |: n.
Does the car have (poor_visibility)? (y/n): |: n.
Sorry, the system cannot identify the problem.
true.
```